

Lesson 6 — Capstone: ASCII Arcade

One lesson: 2 hours 30 minutes · keep this sheet

Name

Class

Date

Cheat sheet

Headers and sources

File	Holds	Think of it as
screen.h	declarations — what exists	the menu
screen.cpp	definitions — how it works	the kitchen

```
// screen.h
#pragma once
void put(int x, int y, char c);

// screen.cpp
#include "screen.h"
static char buffer[WIDTH * HEIGHT]; // private to this file
void put(int x, int y, char c) { ... }
```

#include "mine.h" — quotes for your files
#include <iostream> — brackets for the library
#pragma once — at the top of **every** header

Compile vs link

Error	Step	Cause
use of undeclared identifier 'put'	compile	missing #include
Undefined symbols: "put(int,int,char)"	link	no .cpp defines it, or it is missing from the build

Read **which step** failed. They look similar and are not.

Makefile

```
CXX = clang++
CXXFLAGS = -std=c++17 -Wall
OBJECTS = main.o game.o screen.o

invaders: $(OBJECTS)
→ $(CXX) $(OBJECTS) -o invaders

%.o: %.cpp
→ $(CXX) $(CXXFLAGS) -c $< -o $@
```

→ **must be a REAL TAB**. Spaces give missing separator and no hint why.

\$< = the input · \$@ = the output · -c = compile only

The fixed timestep

```
const double STEP = 1.0 / 60.0;
accumulator += frame_time;
while (accumulator >= STEP) {
    update(STEP); // ALWAYS the same dt
    accumulator -= STEP; // SUBTRACT, do not zero
}
```

- **Clamp frame_time** (≤ 0.25 s) or a long stall becomes a spiral of death.
- Steps never get big → **no tunnelling**.
- Same inputs → same result → **replays and multiplayer are possible**.

Section A — Recall

A1. What goes in a `.h` file and what goes in a `.cpp`? Why split them at all?

[3 marks]

A2. What does `#pragma once` do, and what happens without it?

[2 marks]

A3. Explain the difference between *compiling* and *linking*. Give an error message that belongs to each.

[3 marks]

A4. What problem does a fixed timestep solve that delta time does not?

[2 marks]

A5. Why `accumulator -= STEP` rather than `accumulator = 0`? And why clamp `frame_time`?

[3 marks]

Section B — Predict the output

B1. `STEP` is 1/60. A frame took **0.05 s**. How many times does `update` run, and what is left in the accumulator? Now the next frame takes 0.005 s — how many times then?

[4 marks]

B2. Which step fails, and why?

[3 marks]

```
// game.cpp
#include "screen.h"
void draw() { put(1, 1, '@'); }
```

...and `screen.cpp` is left out of the Makefile's `OBJECTS`.

B3. A student edits `screen.h` and runs `make`. Nothing rebuilds. What is missing from the Makefile, and what will they experience?

[3 marks]

B4. The laptop is suspended for **10 seconds** mid-game. Describe what happens with each version. [4 marks]

```
// version A - delta time
update(frame_time);

// version B - fixed timestep, no clamp
accumulator += frame_time;
while (accumulator >= STEP) { update(STEP); accumulator -= STEP; }

// version C - fixed timestep, clamped to 0.25
```

Section C — Find and fix the bug

C1. make says `Makefile:12: *** missing separator. Stop.` What is wrong, and why is the message so unhelpful? [3 marks]

C2. This compiles, but the linker complains that `WIDTH` is defined more than once. Why — and why does the `const int` version work? [4 marks]

```
// screen.h
#pragma once
int WIDTH = 46;
```

C3. The game runs at roughly half speed. The frame rate is fine. What is wrong? [4 marks]

```
accumulator += frame_time;
if (accumulator >= STEP) {
    game_update(STEP);
    accumulator = 0.0;
}
```

Section D — Build it

This is the capstone. 85 minutes. You may start from the project in `code/` and extend it, or build your own arcade game from scratch.

- ☐ **Checkpoint 1.** Split your lesson 5 Pong into at least `screen.h / screen.cpp` and `main.cpp`. Get it compiling with one long `clang++` command.
- ☐ **Checkpoint 2.** Write a Makefile. Prove it works incrementally: change one `.cpp`, run `make`, and check only that file rebuilds.
- ☐ **Checkpoint 3.** Add `terminal.h / terminal.cpp` so the raw-mode code lives in one place and nothing else mentions `termios`.
- ☐ **Checkpoint 4.** Build the game itself in `game.h / game.cpp`, keeping `main.cpp` to just the loop. Aliens or asteroids, bullets, a player, collisions, a score.
- ☐ **Checkpoint 5.** Replace delta time with a **fixed timestep**, with the accumulator and the clamp.
- ☐ **Checkpoint 6.** Add waves that get harder (with a *floor*), lives, a menu and a game-over screen. Then **change something to make it yours.**

HOW THIS IS MARKED

Not on features. On: does it build with `make` and run; is it genuinely split, with headers declaring and sources defining; can you explain any function I point at; and **did you change something to make it yours?**

Section E — Design challenge

NO RIGHT ANSWER

E1. Your project has four `.cpp` files. **Could you reuse any of them, unchanged, in a completely different game?** Which, and what makes the difference between the reusable ones and the rest?

E2. `screen.cpp` keeps its buffer `static`, so no other file can touch it directly. **Why is that better than making it available everywhere?** What would go wrong if any file could write to it?

E3. Design the file split for a different game — a platformer, say. What files, and what goes in each?

E4. The big one. You have built five games in C++ now. **Design the part that is left over when you take the game out of the game.** What would be in your reusable skeleton? What must always stay specific to one game?

Stretch goals

1. Power-ups dropped by destroyed aliens.
2. A boss wave every fifth wave.
3. Save the high score, surviving a missing or damaged file.
4. A `--debug` flag that draws hitboxes. (Look up `argc` and `argv`.)
5. **Split it further** into `aliens.h` / `aliens.cpp`. Did that help, or is it just more files? Be honest.
6. **Make it yours.**